Data 37711: Machine Learning & AI Part I **Fall 2025**

Homework 2

Lecturer: Victor Veitch Due date: November 6th, 11:59 pm

1. So far in this course we have focused on fitting models $f_\mu(x)$ to approximate $\mathbb{E}[Y|X]$, the conditional mean of the target Y given the input X. Suppose we want to also incorporate a measure of uncertainty in our observations of Y, in the form of a function $f_\sigma(x) = \sqrt{\text{Var}(Y|X=x)}$ (the conditional standard deviation) which models how the observation noise changes with the input X.

   (a) Say we have a neural architecture $g : \mathbb{R}^p \to \mathbb{R}^2$ that takes in a p-dimensional feature vector and returns a 2-dimensional vector of real numbers. Suggest a link function that would be appropriate for converting the output to f_μ and f_σ.

   (b) We could model data of this form as $Y|X \sim \mathcal{N}(f_\mu(X), f_\sigma(X)^2)$. Write down a loss function we could use for model training based on this model.

   (c) We have provided a dataset from taxi trips taken in NYC in `taxi_trips.parquet`. It contains 8 total columns, 7 features and 1 target (`trip_duration`, in minutes). The 7 features (pickup/dropoff coordinates and time-of-day, day-of-week and day-of-year) have already been standardized, so do not expect them to look like raw latitudes and longitudes. Define a multi-layer perceptron in PyTorch or JAX that outputs a conditional mean and standard deviation, and fit this on the taxi trip dataset. Make sure to follow best practices: define a baseline (or multiple) baseline models that you think your model should beat, split your dataset into training/validation/test sets, etc.

   (d) How would we use the model we've just fit to predict the probability that a trip will take less than 45 minutes? The normal distribution assumption is fine.

   (e) Estimate these probabilities on your held-out test set and evaluate them. Report both a proper scoring rule for the predicted probabilities (e.g. the Brier score or the log-loss) and the classification accuracy obtained by thresholding the probability at 0.5.

2. For this problem, you will explore different methods for encoding categorical random variables. We have provided a dataset `retail.csv`, which contains data on retail transactions. We've cleaned up the data a bit so you only have two features: `StockCode` and `CustomerID`, both of which are categorical. (You may need to convert them to categorical data types in your code for the encoding methods to work properly.) The target is `Quantity`, which is a positive integer. Note that `Quantity` is very heavy-tailed (most values are small, but a few are enormous), so think about what loss function and/or target transformation is appropriate.

   (a) Start by exploring the cardinality of the two features. How many unique values are there for each feature? What does this suggest about how you might encode them? Is one-hot encoding feasible? To prepare for the next steps, split the data into training, validation, and test sets. The validation set will allow you to use early stopping during training for the neural network models.

   (b) Try target encoding. For each unique value of the feature, replace it with the mean of the target for that value (from the training set). You can use `TargetEncoder` from the `category_encoders` package. Using an appropriate loss function, fit a model of your choice to the data using the target-encoded features. Evaluate the model on the test set.

   (c) Try a neural network model with an embedding layer for each of the two features. Choose a reasonable embedding dimension. Fit this model to the data. Evaluate the model on the test set. How does it compare to the target encoding model?

   (d) What if we combine the two? That is, use both the target encoded features and the embeddings as input to the neural network. Fit this model to the data. Evaluate the model on the test set. How does it compare to the other two models?

   (e) Try to give some intuition for your results. What is the embedding layer doing? Why might it be useful in this context? Evaluate your findings on the test set.

3. In class, we have discussed an important idea in machine learning: constructing a large set of features and then fitting relatively simple models—such as linear models or linear models composed with simple nonlinear transformations—on top of these features. In particular, we have seen how both the **Random Kitchen Sink** (RKS) model and the **Multi-Layer Perceptron** (MLP) fall under this general paradigm. In this problem, you will explore, implement, and evaluate these two models on a toy dataset.

   A brief reminder on Random Kitchen Sink: Given covariates $x_i \in \mathbb{R}^d$ and responses $y_i \in \mathbb{R}$, the RKS model first maps each covariate into a higher-dimensional feature space using randomly generated sinusoidal features. Specifically, consider h pairs of feature functions $(\phi_j^{\sin}, \phi_j^{\cos})_{j=1}^h$, defined as

$$\phi_j^{\sin}(x_i) = \sin(w_j^T x_i + b_j), \quad \phi_j^{\cos}(x_i) = \cos(w_j^T x_i + b_j),$$

   where $b_j \sim \mathcal{U}[0, 2\pi]$ and $w_j \sim \mathcal{N}_d(0_d, \mathcal{I}_d/\sigma^2)$, with σ^2 controlling the scaling of the random projection. The resulting feature representation is

$$z_i = \left(\phi_1^{\sin}(x_i), \phi_1^{\cos}(x_i), \ldots, \phi_h^{\sin}(x_i), \phi_h^{\cos}(x_i)\right)^\top \in \mathbb{R}^{2h}.$$

   A simple linear model is then fit on these features to predict the response. Because the number of features $2h$ can be large relative to the number of samples, we employ **ridge regression** (i.e., linear regression with L^2 regularization) to prevent overfitting. (In the provided starter code the total number of random features $2h$ is called `D`.)

   (a) **Data visualization:** You are given a pre-generated dataset with training, validation, and testing splits. Begin by visualizing the training data. The dataset corresponds to samples (x_1, x_2, y) and you can be assured that (x_1, x_2) are each bounded between $[-3, 3]$. You can either generate a 2D grid-plot where the y value is represented by a color at different (x_1, x_2) locations, or a 3D plot where we visualize the response y on the z-axis.

   (b) **Random Kitchen Sink with Ridge Regression:** Implement and fit an RKS model to the given dataset with ridge regression. You will need to vary one of the key hyperparameters while keeping others fixed at reasonable defaults. Possible hyperparameters include:

   - the number of random features h,
   - the kernel scale parameter σ, or
   - the ridge regularization strength λ.

   Perform a hyperparameter sweep over your chosen parameter and record both the training and validation losses (MSE). Plot the losses as a function of the hyperparameter (two plots, one for training and one for validation loss), and select the best hyper-param based on the validation loss.

   (c) **1-Hidden Layer Multi-Layer Perceptron:** Next, fit the same dataset using a one-hidden-layer MLP implemented in PyTorch or Jax (the provided starter code is in PyTorch). As before, you will choose one hyperparameter to vary (for example, the hidden layer dimension, learning rate, or batch size) and keep other hyperparameters fixed at default values. Plot the training and validation losses over the sweep and determine the best-performing configuration based on the validation loss.

   (d) **Comparison and visualization:** Using the best-performing hyperparameters from the previous steps, refit both models and report the validation loss (MSE) of each. Discuss which approach performs better and why. The provided `test.csv` is *unlabeled* (it contains only x_1, x_2): produce predictions from both models on it and submit them as two CSV files, `rks_test_predictions.csv` and `mlp_test_predictions.csv`, one prediction per row in the same order as `test.csv`. Finally, visualize the fitted functions from the two models over the input space (we provide an example function to do this). Compare these surfaces with your visualization of the training data from part (a), and qualitatively explain how each model captures the underlying structure of the data.

   Provided resources: You are provided with

   - `train.csv / val.csv / test.csv` the train/validation/test data points (`test.csv` has no `y` column)
   - `dataloader.py` a handy data structure to load the given data
   - `plotting.py` an example plotting function that helps with the visualization in part (d)

- `models.py` skeleton code for implementing the RKS/MLP methods, performing training as well as hyper-parameter sweeping

Note that you should **feel free to implement your own code** (i.e. ignore these helper files) as long as it's in Python and you are using the provided dataset. Make sure to attach your code as part of the assignment submission.